INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE - Second Year

Introduction to Programming Using Java Lab (BCS351)

Lab Assignments (15)

Instructions

- Do one assignment in each lab session, in the order given.
- Write and run the code yourself. Keep the program file and a screenshot or copy of the output for each task.
- Submit the file(s) named in the Submit line of each assignment. Write your name, roll number and the assignment number at the top of every file.

List of assignments

1. Hello World, Variables and Arithmetic
2. Decisions and Loops with Numbers
3. Star and Number Patterns
4. Writing Functions
5. Arrays: Traversal and Counting
6. Arrays: Reverse and Duplicates
7. Counting Steps and Complexity
8. Matrices: Transpose and Row Sums
9. Matrices: Rows and Columns
10. Switch and ArrayList
11. Bubble Sort and Selection Sort
12. Insertion Sort
13. Strings: Methods and Palindromes
14. Characters: Counting and Positions
15. StringBuilder Methods

Assignment 1: Hello World, Variables and Arithmetic

Aim: Print text, store values in variables and calculate with arithmetic operators.

Tasks

1. Print Hello World on one line. Then print Welcome to Java on the next line.

   Sample:

```
Output:
Hello World
Welcome to Java
```

2. Create int roll = 101, double percentage = 84.5, char section = 'B' and String name = "Asha". Print each value on its own line, after the labels Roll:, Percentage:, Section: and Name: in that order.

   Sample:

```
Output:
Roll: 101
Percentage: 84.5
Section: B
Name: Asha
```

3. Create int variables a = 17 and b = 5. Print Sum:, Difference:, Product:, Quotient: and Remainder:, each followed by its result for a + b, a - b, a * b, a / b and a % b.

   Sample:

```
Output:
Sum: 22
Difference: 12
Product: 85
Quotient: 3
Remainder: 2
```

4. Create a constant named PI with the final keyword and the value 3.14. For a circle with radius 7, print Area: followed by the area and Perimeter: followed by the perimeter, using PI.

   Sample:

```
Output:
Area: 153.86
Perimeter: 43.96
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 2: Decisions and Loops with Numbers

Aim: Use if, else if and loops to make decisions and repeat steps with whole numbers.

Tasks

1. Store 27 in an int named n. Print the value of n followed by is even or is odd, using the % operator and an if-else statement.

   Sample:

```
Output:
27 is odd
```

2. Store 83 in an int named marks. Print Grade: followed by A for 90 or more, B for 75 to 89, and C for less than 75. Use an else-if ladder.

Sample:

```
Output:
Grade: B
```

3. Store 2024 in an int named year. A leap year is divisible by 4 but not by 100, or divisible by 400. Print Leap year for a leap year, otherwise Not a leap year.

   Sample:

```
Output:
Leap year
```

4. Use a for loop to find the factorial of 5. Print Factorial of 5 is followed by the value.

   Sample:

```
Output:
Factorial of 5 is 120
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 3: Star and Number Patterns

Aim: Use a loop inside another loop to print stars and numbers in rows.

Tasks

1. Print a square of stars with 4 rows and 4 columns. Use one loop inside another loop.

   Sample:

```
Output:
****
****
****
****
```

2. Print a right triangle of stars with 4 rows. Row 1 has one star, row 2 has two stars, and so on.

   Sample:

```
Output:
*
**
***
****
```

3. Print a number triangle with 4 rows. Row 1 prints 1. Row 2 prints 1 2. Row 3 prints 1 2 3, and so on. Put a space after each number.

   Sample:

```
Output:
1
1 2
1 2 3
1 2 3 4
```

4. Print an upside-down triangle of stars with 4 rows. Row 1 has four stars, row 2 has three stars, and so on, down to one star in row 4.

   Sample:

```
Output:
****
***
**
*
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 4: Writing Functions

Aim: Write functions with parameters and return values, and call them from main.

Tasks

1. Write a function fact(int n) that returns n factorial, using a loop, and a function nCr(int n, int r) that returns n C r, using fact. In main, print 5C2 = followed by the result of nCr(5, 2).

   Sample:

```
Output:
5C2 = 10
```

2. Write fact(int n) that returns n factorial using a loop, and nPr(int n, int r) that returns n P r using fact. In main, print 5P2 = followed by the result of nPr(5, 2).

   Sample:

```
Output:
5P2 = 20
```

3. Write maxOf(int a, int b) that returns the larger of its two int values. In main, print Larger: followed by the result of maxOf(14, 9).

   Sample:

```
Output:
Larger: 14
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 5: Arrays: Traversal and Counting

Aim: Create an array, visit every element with a loop, and add up or count the values.

Tasks

1. Create an int array named arr with the values 3, 4 and 5. Use a for loop with arr.length to print one line for each element: the label Index, the index number, a colon and the element value.

   Sample:

```
Output:
Index 0: 3
Index 1: 4
Index 2: 5
```

2. Create an int array with the values 12, 45, 7, 33 and 21. Use a loop to find the sum and the largest value. Print Sum: followed by the sum, then Largest: followed by the largest value.

   Sample:

```
Output:
Sum: 118
Largest: 45
```

3. Create an int array with the values 5, 12, 2, 14 and 7. Use a loop to count the values greater than 10. Print Count greater than 10: followed by the count.

   Sample:

```
Output:
Count greater than 10: 2
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 6: Arrays: Reverse and Duplicates

Aim: Print an array in reverse order, find the smallest value, and check an array for repeated values.

Tasks

1. Create an int array with the values 10, 20, 30, 40 and 50. Print the values in reverse order, one value on each line, starting with the last value.

   Sample:

```
Output:
50
40
30
20
10
```

2. Create an int array with the values 2, 5, 1, 8, 3 and 4. Use a loop to find the smallest value and its index. Print Smallest: followed by the value, then the words at index and the index.

   Sample:

```
Output:
Smallest: 1 at index 2
```

3. Create an int array with the values 4, 9, 2, 9 and 7. Use nested loops to check for a repeated value. Print Repeated value: followed by it, or print No repeated value if none is repeated.

   Sample:

```
Output:
Repeated value: 9
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 7: Counting Steps and Complexity

Aim: Count the steps a program takes, and write its time complexity as a comment.

Tasks

1. Use one loop that runs from 1 to 10, and count its steps. Above the loop, write a comment with its time complexity. Print Steps: followed by the number of steps.

   Sample:

```
Output:
Steps: 10
```

2. Use two loops, one inside the other, that both run from 1 to 10. Count how many times the inner loop body runs. Above the loops, write a comment with their time complexity. Print Steps: followed by the total.

   Sample:

```
Output:
Steps: 100
```

3. Store 1024 in an int named n. Use a while loop that halves n until n is 1, and count the divisions. Above the loop, write a comment with its time complexity. Print Halving steps: followed by the count.

   Sample:

```
Output:
Halving steps: 10
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 8: Matrices: Transpose and Row Sums

Aim: Create two-dimensional arrays, visit them row by row, and build a new matrix from one.

Tasks

1. Create a matrix named m with 3 rows and 4 columns. Fill it with the values 1 to 12, row by row, using nested loops. Print the matrix one row per line, with a space after each number.

   Sample:

```
Output:
1 2 3 4
5 6 7 8
9 10 11 12
```

2. Create the 3 by 3 matrix with the rows 1 2 3, 4 5 6 and 7 8 9. Create its transpose (rows and columns swapped). Print the transpose one row per line, with a space after each number.

   Sample:

```
Output:
1 4 7
2 5 8
3 6 9
```

3. Use the 3 by 3 matrix with the rows 1 2 3, 4 5 6 and 7 8 9. Print the sum of each row, one line per row, in the form Row 1 sum: followed by the total.

   Sample:

```
Output:
Row 1 sum: 6
Row 2 sum: 15
Row 3 sum: 24
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 9: Matrices: Rows and Columns

Aim: Visit the rows and columns of a matrix with nested loops to print, add and count its values.

Tasks

1. Use the 3 by 3 matrix with rows 1 2 3, 4 5 6 and 7 8 9. Print First row: and its values, then Last row: and its values, with a space between values.

   Sample:

```
Output:
First row: 1 2 3
Last row: 7 8 9
```

2. Use the 3 by 3 matrix with the rows 1 2 3, 4 5 6 and 7 8 9. Print the sum of each column, one line per column, in the form Column 1 sum: followed by the total.

   Sample:

```
Output:
Column 1 sum: 12
Column 2 sum: 15
Column 3 sum: 18
```

3. Use the 3 by 3 matrix with the rows 1 2 3, 4 5 6 and 7 8 9. Count the values greater than 5 and print Values greater than 5: followed by the count.

   Sample:

```
Output:
Values greater than 5: 4
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 10: Switch and ArrayList

Aim: Choose a branch with a switch statement, and store and change a list of numbers in an ArrayList.

Tasks

1. Store 3 in an int named day. Use a switch statement to print the name of the day: 1 is Monday, 2 is Tuesday and 3 is Wednesday. For any other number, print Invalid day.

   Sample:

```
Output:
Wednesday
```

2. Create an ArrayList of Integer named list. Add the values 10, 20, 30 and 40. Print the whole list. Then print Size: followed by its size, and Item at index 2: followed by the value at index 2.

   Sample:

```
Output:
[10, 20, 30, 40]
Size: 4
Item at index 2: 30
```

3. Create an ArrayList of Integer named list with the values 10, 20, 30 and 40. Use set to change index 1 to 25, and remove to delete index 3. Print the list and Size: followed by its size.

   Sample:

```
Output:
[10, 25, 30]
Size: 3
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 11: Bubble Sort and Selection Sort

Aim: Sort small arrays with bubble sort and selection sort, and count the swaps.

Tasks

1. Create the int array 5, 1, 4, 2, 8. Sort it in increasing order with bubble sort. Print Sorted: followed by the sorted values in square brackets, separated by commas.

Sample:

```
Output:
Sorted: [1, 2, 4, 5, 8]
```

2. Create the int array 64, 25, 12, 22, 11. Sort it in increasing order with selection sort. Print Sorted: followed by the sorted values in square brackets, separated by commas.

   Sample:

```
Output:
Sorted: [11, 12, 22, 25, 64]
```

3. Use the array 5, 1, 4, 2, 8 and sort it with bubble sort. Count the swaps (each time two values change places). Print Swaps: followed by the count.

   Sample:

```
Output:
Swaps: 4
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 12: Insertion Sort

Aim: Sort small arrays with insertion sort and count how many values are shifted.

Tasks

1. Create the int array 7, 3, 5, 1. Sort it in increasing order with insertion sort. Print Sorted: followed by the sorted values in square brackets, separated by commas.

   Sample:

```
Output:
Sorted: [1, 3, 5, 7]
```

2. Create the int array 14, 3, 9, 21, 6. Sort it in increasing order with insertion sort. Print Sorted: followed by the sorted values in square brackets, separated by commas.

   Sample:

```
Output:
Sorted: [3, 6, 9, 14, 21]
```

3. Create the int array 6, 5, 4, 3, 2, 1. Sort it in increasing order with insertion sort. Count the shifts (each time one value moves one place to the right). Print Shifts: followed by the count.

   Sample:

```
Output:
Shifts: 15
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 13: Strings: Methods and Palindromes

Aim: Use common String methods, reverse a word, check a palindrome and print substrings.

Tasks

1. Use the word Programming. Print Length: followed by its length, Position of g: followed by the position of the first g (counting from 0), and Upper case: followed by the word in capital letters, using toUpperCase().

Sample:

```
Output:
Length: 11
Position of g: 3
Upper case: PROGRAMMING
```

2. Reverse the word Java. Build the reversed word one letter at a time, using charAt and + in a loop. Print Reversed: followed by the result.

   Sample:

```
Output:
Reversed: avaJ
```

3. Use the word racecar. Check whether it is a palindrome, which means it reads the same from both ends. Print the word followed by is a palindrome, or followed by is not a palindrome.

   Sample:

```
Output:
racecar is a palindrome
```

4. Use the word abc. Print every substring of it, one on each line, using substring(). For each starting letter from left to right, print its parts from shortest to longest.

   Sample:

```
Output:
a
ab
abc
b
bc
c
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 14: Characters: Counting and Positions

Aim: Use charAt to count vowels and a letter, and print each letter with its position.

Tasks

1. Store programming in a String named s. Count its vowels (a, e, i, o and u). Print Vowels: followed by the count.

   Sample:

```
Output:
Vowels: 3
```

2. Store programming in a String named s. Count how many times the letter g appears. Print Count of g: followed by the count.

   Sample:

```
Output:
Count of g: 2
```

3. Store hello in a String named s. Print each letter on its own line as its position, a colon, a space and the letter. Positions start at 0.

   Sample:

```
Output:
0: h
1: e
2: l
3: l
4: o
```

Submit: One .java file per task (class Main) and a copy of its output.

Assignment 15: StringBuilder Methods

Aim: Build and change text with StringBuilder, using append, insert and deleteCharAt.

Tasks

1. Create a StringBuilder that starts with the text Hello. Append the text " World" to it. Print the StringBuilder, then print Length: followed by its length.

   Sample:

```
Output:
Hello World
Length: 11
```

2. Use a StringBuilder. Append each number from 1 to 5, followed by a comma. Then remove the last comma with deleteCharAt. Print the result.

   Sample:

```
Output:
1,2,3,4,5
```

3. Create a StringBuilder that starts with the text Java. Use insert to add the text Hello followed by a space at index 0. Print the StringBuilder.

   Sample:

```
Output:
Hello Java
```

Submit: One .java file per task (class Main) and a copy of its output.